

DJI Storefront — Search Experience PRD

Status: Shipped (prototype) **Last updated:** 2026-05-01 **Owner:** Storefront / Search **Source of truth:** [src/catalog/searchEngine.ts](#) , [src/catalog/catalog.ts](#) , [src/components/SearchOverlay/](#)

1. Background

The Agentic Commerce prototype runs against a real DJI catalog of ~250 SKUs scraped from JB Hi-Fi (`dji_products_tagged_v6.csv`). The catalog mixes flagship hardware (drones, action cameras, gimbals, mics) with bundles ("Fly More Combo", "Adventure Combo") and deeply nested accessories (batteries, ND filters, mounts, cages, charging cases, propellers).

Search is the primary discovery surface across three flows:

1. **Search overlay** — full-screen dropdown launched from the top-nav magnifier, with two columns (Recent searches / Suggestions on the left, ranked product grid on the right) and an assistant promo footer.
2. **PLP `?q=`** — typing in the overlay and hitting Enter routes to `/products?q=<query>` which renders the same ranked result set as a full grid with filters.
3. **Assistant tool calls** — the OpenAI agent uses the same engine via `searchProducts(query)` when it needs to surface SKUs to the user.

All three call into a single in-memory engine built once at module load. There is no server, no Algolia, no Elasticsearch — the entire experience runs client-side in <50 ms per query at 250 docs.

2. Goals

Primary

- **Right model wins.** A query that names a model line ("action 3", "mavic 4 pro", "mini 5") returns SKUs from *that* model line, not lexically-similar siblings (Action 5, Mavic 3, Mini 4).
- **Right intent wins.** A query like "accessories for action 5 camera" returns Action 5 accessories — not Action 3 hosts, Mic 3 transmitters, or random combos with "5" buried in spec copy.
- **Casual phrasing works.** "drone with low light", "vlog mic", "filter set for mini 4 pro" should land sensible results without the user typing a precise SKU.
- **Typo tolerance.** "mavik 4" should still find Mavic 4. Single-letter typos shouldn't tank the experience.
- **Fast.** Under 50 ms per query end-to-end, including the React re-render of the overlay grid. No perceivable lag while typing.

Non-goals

- Personalisation / ranking by user history.
- Server-side search infrastructure.

- Faceted query parsing ("price < 500 and category = drone") — filters live on the PLP, not in the keyword box.
- Spell correction beyond single-token Levenshtein swaps.

3. User-facing principles

Principle	What it looks like in the product
AND, not OR	Typing two words means you want results matching <i>both</i> . Single-word leaks (a hit in just one weak field) don't surface.
Phrase awareness	"Action 3" is one concept, not two. The engine treats consecutive model-line tokens as a phrase that must occur together.
Strong fields beat weak fields	A name match outranks a category match outranks a tag match outranks a description match.
Synonyms, not stemming	"vlog" expands to "vlogging / youtube"; "drone" expands to "quadcopter / uav / aerial". We don't aggressively stem — that produces more false positives than it solves.
Typos cost confidence	Fuzzy hits count for half a normal hit, so they only float to the top when nothing exact lands.
Empty results suggest	A zero-result query gets a "Did you mean: ..." prompt, but only when the corrected query actually has hits.

4. Architecture overview



The index is built **once** (catalog load is module-scoped) and shared across the React tree via `CatalogContext`. Each query allocates only the per-query scratch arrays (`Float64Array(scores)` , `Uint8Array(included)` , expansion maps).

5. The retrieval pipeline

5.1 Normalisation

`normalize(value)` lowercases, strips diacritics (NFD + combining-mark removal), turns curly quotes into straight quotes, and collapses anything non-alphanumeric to a single space. This keeps alphanumeric model tokens like `4k` , `2s` , `rs2` , `5135` intact.

5.2 Tokenisation

`tokenize(value)` runs `normalize` then splits on spaces and drops:

- Stop-words:** the / a / an / and / or / for / with / of / to / in / on / at / by / is / are / be / from / as . Real query intent ("battery for mini 4") loses nothing useful.

- **Single-char alpha tokens** ("a", "b") — but **digits are kept**. This was a deliberate fix: dropping single-digit tokens collapsed "Mini 5", "Mini 4", "Mini 3" into a single bucket and let older model years out-rank the queried one.

5.3 Synonym expansion

Each token is looked up in a hand-curated `SYNONYMS` map (~25 entries across content/use-case, audio, drones, stabilisation, cameras, water, tier, FPV, travel). Expansion is unidirectional per call: we walk the original tokens once and append related terms. This avoids accidental "audio → mic → microphone → audio" loop expansion.

The output is a `QueryPlan` :

- `originalTokens` — deduped non-stopword tokens in input order. Used for the AND-gate.
- `expansionsByToken` — map from each original token to its full expansion list (the original first, then synonyms). The gate is satisfied per *original* token; synonyms are alternative spellings of the same intent, not additive evidence.
- `expandedTokens` — flat union of all expansions (legacy, used for fuzzy candidate dedup).

5.4 Indexing (per-doc fields)

For each `CatalogProduct` we build six token bags:

Field	Contents	Score weight
<code>nameTokens</code>	Product title	6 (<code>partialName</code>)
<code>categoryTokens</code>	Category string ("Action cameras", "Lens filters", etc.)	5
<code>tagTokens</code>	<code>useCaseTags</code> ∪ <code>capabilities</code> ∪ <code>subtypes</code> ∪ <code>primaryActivities</code> ∪ <code>series</code> ∪ <code>accessoryRole</code> (with underscores stripped to spaces)	4
<code>brandTokens</code>	<code>brand</code> (always "DJI" today)	3
<code>compatibleTokens</code>	<code>compatibleWithModels</code> (word-tokenised) ∪ <code>compatibleWithType</code> (collapsed to atomic tokens — see below)	3
<code>descriptionTokens</code>	<code>shortDescription</code> + first 4 feature blocks	2
<code>nameNormalized</code>	Full title string for the exact-phrase bonus	+10 stacked

Plus two **ordered** fields powering the phrase-proximity gate:

Field	Why ordered
<code>nameTokenSequence</code>	Preserves position so we can ask "does <code>action</code> precede <code>3</code> within N tokens in the title?"
<code>compatibleModelTokenSequence</code>	Same question for the compatibility-models list (so accessories whose own title doesn't repeat the host model still satisfy phrase requirements).

There's also an inverted **postings** map (token → `docIdx[]`) and a **tokensByLength** map used to speed up Levenshtein candidate selection.

5.5 The AND-gate

For each doc, every original token must produce *at least one* hit somewhere — otherwise the doc is dropped. Without this, a doc that hit only the literal `"Accessories"` category for query "drone accessories action camera" would float in for any query mentioning the word.

Per token, we score the **single best field hit** across the doc's bags (we don't double-count "drone" appearing in both name and description — strongest field wins).

5.6 Fuzzy fallback

If no exact expansion lands for a token, we fall back to Levenshtein candidates within distance ≤ 2 , only for tokens of length ≥ 4 (shorter tokens have edit distance too lax to be useful). Candidates are pre-bucketed by length so we skip impossible matches before paying for the matrix. Fuzzy hits are scored at **half weight** (`FUZZY_MULTIPLIER = 0.5`) so they only surface when nothing exact lands.

5.7 Phrase / proximity gate

Bigrams in the original query where the right token looks like a model digit (`/^\d+[a-z]*$/` — covers `3` , `5` , `4k` , `2s`) are promoted to **phrase requirements**. The doc must contain those tokens in order, with at most **4 intervening tokens**, in either:

- the title (`nameTokenSequence`), or
- the compatibility-models list (`compatibleModelTokenSequence`).

The gap of 4 accommodates list-style titles like `"Osmo Action 5 Pro / 4 / 3"` (which tokenises to `osmo action 5 pro 4 3` — gap of 4 between `action` and `3`). Going wider risks pulling in incidental co-occurrences from long descriptive titles.

This runs **before** the AND-gate so phrase-failing docs are rejected cheaply.

5.8 Exact-phrase bonus

After the per-token AND-gate clears, we check whether the entire normalised query appears as a substring in the title. If yes, +10. This is a tie-breaker that rewards docs where the user literally typed the product name.

5.9 Final ranking

```
score desc → rating desc → reviewCount desc → docIdx asc (stable)
```

Equal-score products fall back to social proof (rating then review count) so high-confidence SKUs win ties.

6. Two specific defences (the recent fixes)

These are documented separately because they're the answer to "why doesn't this leak?" questions.

6.1 Compatibility-type token collapse (`collectCompatibleText`)

The CSV ships compatibility *types* like `compatible_with_type=["drone","action_camera"]`. Naïvely tokenising `action_camera` produces `["action", "camera"]` — and that single loose `"action"` token would clear the AND-gate for any query containing "action", flooding results with random batteries / chargers / cables tagged `action_camera`.

Fix: collapse the underscore so each type stays a single atomic token (`actioncamera`). Word-level matches like `"action"` from a user query must hit the accessory through name / category / tags instead.

```
// src/catalog/searchEngine.ts
const typeText = p.compatibleWithType.map((t) => t.replace(/_/g, " ")).join(" ");
```

6.2 Numeric-token field guard (`scoreTokenAgainstDoc`)

Numeric tokens (`/^\d+$/` — `3`, `5`, `7`) carry real signal in product names ("Mini 5", "Action 5", "RS 4"). They carry **no** signal in description prose, where they appear as:

- footnote markers (`"[5] action 3's innovative quick-release..."`)
- spec fragments (`"energy: 11.5 Wh"`, `"5° to 40° C"`)
- counts (`"5-axis stabilisation"`)

Before this guard, a query like "action 5 accessories" leaked Action 3 hosts, Mic 3 transmitters, and Neo 2 batteries because each had a stray `5` somewhere in its description. The fix: digit tokens can only satisfy via name, category, tag, or compatible — never description (or brand, where digits never legitimately appear).

```
const numeric = isNumericToken(token);
if (doc.nameTokens.has(token)) consider(SCORE.partialName);
if (doc.categoryTokens.has(token)) consider(SCORE.category);
if (doc.tagTokens.has(token)) consider(SCORE.tag);
if (!numeric && doc.brandTokens.has(token)) consider(SCORE.brand);
if (doc.compatibleTokens.has(token)) consider(SCORE.compatible);
if (!numeric && doc.descriptionTokens.has(token)) consider(SCORE.description);
```

6.3 Phrase-proximity gate (the biggest single quality jump)

The two fixes above weren't enough on their own. Even with digit tokens locked out of description, a query like "action 3 accessories" still pulled in `Mavic 3 Fly More Kit`, `SmallRig Carrying Bag for DJI Osmo Pocket 3`, and `DJI Mic 3 Transmitter` — all docs with "3" in their *title* and "action" slipping in via description copy ("action shots", "take action...").

The root cause: the AND-gate treats "action" and "3" as fully independent tokens. Per-token semantics can't express "these tokens belong together."

The fix is a **bigram phrase requirement** detected from the query, satisfied by ordered proximity in the doc's title or compatibility list.

Query	Bigram	Promoted?	Why
action 3	action 3	✓	right token is digit → model-line phrase
mavic 4 pro	mavic 4, 4 pro	✓ / ✗	only <code>mavic 4</code> (right=digit)
osmo pocket 3	osmo pocket, pocket 3	✗ / ✓	only <code>pocket 3</code>
air 2s	air 2s	✓	<code>2s</code> matches <code>\d+[a-z]*</code>
4 3	4 3	✗	both numeric — ambiguous
vlog mic	vlog mic	✗	no digit on either side
drone with low light	(after stop-words) drone low, low light	✗	no digit

Phrase satisfaction allows synonym expansions on both sides (so `drone 4` queries can be satisfied by docs with `quadcopter near 4`), and checks both the title and the `compatibleWithModels` token list so accessories whose title doesn't repeat the host model still match.

7. Did-you-mean

When a query returns zero results, we run a per-token spelling-correction pass:

1. For each token in the query, if it already exists in the catalog vocab, keep it.
2. Otherwise, find the closest Levenshtein candidate within distance 2.
3. Among ties, prefer the candidate with the larger posting list (more frequent → more likely the user's intent).
4. Reassemble. If the corrected query differs from the original AND re-running the engine on the correction yields hits, surface "Did you mean: ?".

This guards against showing a "fix" that's also empty.

8. UX & UI surfaces

8.1 Search overlay (`SearchOverlay.tsx`)

- **Trigger:** magnifier in the top-nav. The overlay replaces the page's nav while open.
- **Layout:** dark replacement header (brand wordmark left, centered search field flanked by `1fr` rails, close X right), followed by a two-column body (Recent / Suggestions left, Bestsellers / Top results right) and an optional Assistant promo footer (suppressed in `basic-website` mode for the bare e-com baseline).
- **Centred field:** the input row uses `grid-template-columns: 1fr auto 1fr` so the 600px field sits in the true horizontal centre regardless of viewport width.
- **Debounce:** 300ms between last keystroke and the live search; clear is instant (snaps debounced query to empty so the overlay flips back to bestsellers immediately).
- **Min query length:** 2 chars before live results fire (single-letter queries dump too many false positives).
- **Highlighting:** matched substrings get `<mark class="search-overlay__highlight">` styling — semi-bold, no background tint. Sorted longest-first so `mini5` highlights `mini5`, not just `mini`.
- **Bestsellers fallback:** when there's no query, the right column shows non-accessory non-bundle SKUs sorted by rating then review count. Top-up from `featuredProducts` if the rated pool is too small.
- **Empty-state:** "No matches for {query}" with optional "Did you mean: ..." CTA.
- **View all:** when results exist, a "View all N results →" CTA submits the query as a PLP `?q=` route.

8.2 PLP `?q=` (`ProductListingPage.tsx`)

- Reads the `searchQuery` route param, calls `searchProducts(query)`, renders the same ranked results in the page grid.
- Filters (category, tier, accessory-only) compose with the keyword search.

8.3 Recent searches

Pre-seeded for the prototype demo (`recentSearches.ts`). Clicking a recent entry submits it as a fresh keyword search (lands on the PLP) rather than dispatching to the assistant — keeps the basic search hygienic.

9. Worked examples

These are the canonical regression cases the engine is verified against.

Query	Expected behaviour	Why it works
accessories for action 5 camera	Returns the 3 SmallRig / Freewell SKUs explicitly tagged for Action 5.	Phrase gate requires action near 5 ; numeric guard kills description-only 5 matches.
action 3 accessories	Returns Action 3 Adventure Combo + SmallRig cage/release that list "/4/3" in the title.	Phrase gate needs action near 3 ; Mavic 3 / Pocket 3 / Mic 3 fail because their titles don't contain action .
mavic 4 pro	Mavic 4 Pro SKUs and Mavic 4 Pro accessories.	mavic 4 phrase requirement; Mavic 3 SKUs fail (no 4 near mavic).
mini 5	DJI Mini 5 Pro + accessories whose compat list names Mini 5 Pro.	Phrase satisfied by name or compatibility models.
mini 4 pro filter	Freewell / PolarPro filter sets explicitly for Mini 4 Pro.	mini 4 phrase narrows to Mini 4 Pro; "filter" anchors the accessory type.
osmo pocket 3	Pocket 3 SKUs and Pocket 3 accessories.	pocket 3 phrase.
mic 3	Only DJI Mic 3 SKUs.	mic 3 phrase excludes Mic 2 SKUs (no 3 near mic).
rs 4 gimbal	DJI RS 4 SKUs only.	rs 4 phrase excludes RS 3 / RS 2.
drone with low light	Pocket 3 / Mini 5 Pro Filter / various low-light-tagged SKUs.	No phrase gate fires (no model digit in query); all tokens hit through tag/category/description.
vlog mic	DJI Mic 2 / Mic 3 SKUs.	Synonym expansion hits the curated vlog / vlogging capability tag.
mavik 4 (typo)	Mavic 4 SKUs.	Levenshtein 1 from mavik → mavic ; phrase gate applies on the corrected token.

10. Performance budget

Stage	Target	Actual at 250 docs
Index build (one-shot at module load)	<100 ms	~30 ms
Per-query end-to-end	<50 ms	<5 ms warm
React grid re-render	<16 ms (1 frame)	within budget
Debounce window (UX-tuned)	300 ms	—

Memory is dominated by the per-doc token sets — bounded at small constants per doc and well under any practical limit at this catalog size.

11. Future work

Documented for future PRs; **not** in scope for the current prototype.

1. **Bigram vocabulary, data-driven phrase detection.** Today phrase promotion fires on any `<word>` `<digit>` bigram. A pre-built "title bigram set" would let us promote *any* bigram that exists as a common product phrase ("fly more", "creator combo", "pro standard"), not just digit-suffixed ones.
 2. **Field-aware position scoring.** Tokens that match earlier in the title (the SKU's lead noun) could outrank those that match in trailing descriptors.
 3. **Faceted intent extraction.** Parse cues like "under \$500", "for cycling", "waterproof" into structured filters before the keyword pass.
 4. **Personalised re-ranking.** Mix in recent views / purchases as a small prior over the rating tie-breaker.
 5. **Server-side index.** At 2,500+ SKUs the in-memory build starts to bite. A precomputed JSON index served from a CDN would extend the same architecture without changing the engine API.
-

12. Files of interest

File	Role
<u>src/catalog/searchEngine.ts</u>	The engine. All retrieval / ranking / phrase logic lives here.
<u>src/catalog/catalog.ts</u>	CSV → <code>CatalogProduct[]</code> normalisation, tag derivation, bundle/accessory classification, series inference. Builds the index.
<u>src/catalog/CatalogContext.tsx</u>	React context that exposes <code>searchProducts(query)</code> to the tree.
<u>src/components/SearchOverlay/SearchOverlay.tsx</u>	The overlay UI, debounce, suggestion list, did-you-mean rendering.
<u>src/components/SearchOverlay/SearchOverlay.css</u>	Layout (1fr-auto-1fr centre-aligned field), responsive grid, dark header, two- column body.
<u>src/pages/ProductListingPage/ProductListingPage.tsx</u>	PLP that reads <code>?q=</code> and renders the same ranked results in grid form with filters.
<u>dji_products_tagged_v6.csv</u>	Source data — one row per SKU, with curated <code>capabilities</code> / <code>subtypes</code> / <code>compatible_with_models</code> / etc.